

Modelo de datos optimizado — Polizing (v3)

Contexto

Modelo rediseñado a partir del esquema actual (`prisma/schema.prisma`), aplicando: aplanado de tablas puente innecesarias (1:1 / 1:N), eliminación de campos calculados (3NF), uso de enums para valores cerrados, auditoría por actor (quién modificó + cuándo) y normalización de relaciones de pagos y siniestros como 1:N reales.

La **v3** incorpora las correcciones recibidas del corrector (ver §6): catálogo de coberturas dependiente del tipo de seguro, categoría de seguro, separación del flag de lectura de siniestros como tabla aparte, enum para método de pago, expansión moderada de estados de siniestro y documentación explícita de campos clave (`prima_mensual` , `siniestros.numero`).

1. Decisiones de diseño aplicadas

1.1 — Aplanar relaciones 1:1 y 1:N que vivían en tablas puente

Tabla puente eliminada	Naturaleza real	Solución
<code>poliza_cliente</code>	1 cliente : N pólizas	<code>polizas.cliente_id</code>
<code>poliza_empresa</code>	1 aseguradora : N pólizas	<code>polizas.aseguradora_id</code>
<code>tipo_poliza</code>	1 tipo : N pólizas	<code>polizas.tipo_seguro_id</code>
<code>siniestros_poliza</code>	1 póliza : N siniestros	<code>siniestros.poliza_id</code>
<code>pagos_polizas</code>	1 pago : N pólizas	<code>polizas.pago_id</code>

Solo quedan relaciones N:M reales si aparecen. En este modelo aparece **una**: cobertura ↔ tipo de seguro se modela mediante el catálogo `coberturas` (ver §1.4).

1.2 — Discriminador explícito de cliente

`clientes.tipo` (enum: `corporativo` | `persona` , NOT NULL). Las subtablas `clientes_corporativos` y `clientes_no_corporativos` se mantienen para los campos exclusivos.

1.3 — Estados y categorías como enum

`PolizaEstado` , `SiniestroEstado` , `PagoEstado` , `ClienteEstado` , `DocumentoTipo` , `CategoriaSeguro` , `MetodoPago` , `Rol` .

1.4 — `polizas.cobertura` como FK al catálogo `coberturas` (no enum)

La versión anterior modelaba `polizas.cobertura` como un enum global con valores de seguro de auto (`responsabilidad_civil` | `terceros_completo` | `todo_riesgo` | `basica` | `integral`). Esto fallaba para pólizas de vida, hogar, ART, etc., donde esos valores no aplican.

La **v3** reintroduce una tabla catálogo `coberturas` (`id` , `tipo_seguro_id` , `nombre` , `descripcion`) . Cada `tipos_seguro` define qué coberturas tienen sentido para él (auto: `responsabilidad_civil/terceros_completo/todo_riesgo`; vida: `muerte/invalidez/ITP`; hogar:

incendio/robo/responsabilidad_civil; etc.). `polizas.cobertura_id` apunta al catálogo. El enum `CoberturaTipo` queda eliminado.

1.5 — Reformular `pagos.monto` y vínculo con pólizas

Como ahora **1 pago agrupa N pólizas**, `pagos.monto` queda como el **total** de la operación (capturado al momento del pago). El detalle por póliza se reconstruye uniendo `polizas` por `pago_id` (con su `prima_mensual`). Se mantiene `pagos.cliente_id` porque el pago corporativo lo inicia un cliente desde su nacimiento (un batch agrupa pólizas de un mismo cliente, y puede existir en estado pendiente antes de tener pólizas vinculadas) — ver §6 para la justificación frente a la observación del corrector.

1.6 — Auditoría por actor

Tablas de negocio (`clientes` , `polizas` , `siniestros` , `pagos` , `empresas_aseguradoras`) llevan:

- `modificado_por_id` (FK → `usuarios.id` , nullable): quién realizó el último cambio.
- `modificado_en` (Timestamptz, nullable): cuándo ocurrió ese cambio.

1.7 — Moneda

`polizas` y `pagos` reciben `moneda CHAR(3) DEFAULT 'ARS'` .

1.8 — Metadata real para documentos

`siniestro_documentos` recibe `url` , `mime_type` . `tamano` pasa a `tamano_bytes` (Int).

1.9 — `tipos_seguro` con `id` artificial + categoría

`id` `autoincrement` como PK; `nombre` queda `@unique` . Se agrega `categoria` (enum `CategoriaSeguro` : `auto` | `vida` | `hogar` | `salud` | `comercio` | `art` | `agricola` | `otros`) para permitir filtros y agrupaciones por rama del seguro. Sin esa columna, dos tipos solo se distinguían por nombre libre.

1.10 — Constraints temporales (CHECK SQL en migración)

- `polizas` : `fecha_fin_vigencia >= fecha_inicio_vigencia`
- `siniestros` : `fecha_reporte >= fecha_ocurrencia`

1.11 — Roles como enum

Se elimina la tabla `roles` . `usuarios.rol` pasa a enum `Rol` .

1.12 — Lectura de siniestros fuera de la entidad

El flag `siniestros.leido` mezclaba estado de UI (por-usuario) con el modelo de dominio. Si hay varios usuarios siguiendo un siniestro, "leído para quién" no tiene respuesta. Se introduce la tabla `siniestro_lecturas` (`siniestro_id` , `usuario_id` , `leido_en`) con PK compuesta. Que un usuario haya leído el siniestro se calcula desde la aplicación (`exists` en la tabla). `siniestros` ya no tiene el campo.

1.13 — Método de pago como enum

`pagos.metodo_pago` deja de ser `String?` y pasa a `MetodoPago?` (`transferencia` | `debito_automatizado` | `tarjeta_credito` | `tarjeta_debito` | `efectivo` | `mercadopago` | `cheque` |

otro). Mantiene nullable porque un pago puede estar pendiente antes de materializarse y, en ese estado, no tiene método asignado. Esto evita inconsistencias y simplifica reportes por método.

1.14 — siniestros.numero documentado

numero (@unique) es el **identificador visible al cliente** (formato SIN-NNNNN , generado por la aplicación con un prefijo + secuencia anual al crear el siniestro). Es distinto del id interno (Int autoincrement de PostgreSQL) y es el que se comunica al asegurado por WhatsApp y se imprime en cualquier comunicación externa. La generación está centralizada en el backend para garantizar unicidad y formato.

1.15 — polizas.prima_mensual documentado

prima_mensual (Decimal 15,2) representa **la cuota mensual fija** que el cliente abona para mantener la póliza vigente. Se captura al momento de la emisión y queda congelada en el período de vigencia (no varía mensualmente; cambios de prima requieren renovación). El monto del pago que cubre la póliza no necesariamente es $12 \times prima_mensual$ — depende del período cubierto y de si hay descuentos por pago anual.

2. Modelo de tablas final

2.1 — clientes

Campo	Tipo	Constraints
id	Int	PK, autoincrement
tipo	enum ClienteTipo	NOT NULL
email	String	nullable
telefono	String	nullable
direccion	String	nullable
estado	enum ClienteEstado	NOT NULL, default activo
fecha_alta	Timestamptz	default now()
modificado_por_id	Int	FK → usuarios.id SetNull, nullable
modificado_en	Timestamptz	nullable

Índices: tipo , estado .

2.2 — clientes_corporativos

Campo	Tipo	Constraints
cliente_id	Int	PK, FK → clientes.id cascade
cuit	String	@unique, NOT NULL
razon_social	String	NOT NULL

contacto_nombre	String	nullable
-----------------	--------	----------

2.3 — clientes_no_corporativos

Campo	Tipo	Constraints
cliente_id	Int	PK, FK → clientes.id cascade
dni	String	@unique, NOT NULL
nombre	String	NOT NULL
apellido	String	NOT NULL

2.4 — empresas_aseguradoras

Campo	Tipo	Constraints
id	Int	PK, autoincrement
razon_social	String	NOT NULL
cuit	String	@unique, NOT NULL
telefono	String	nullable
email	String	nullable
modificado_por_id	Int	FK → usuarios.id SetNull, nullable
modificado_en	Timestamptz	nullable

Eliminados: color_hex , direccion , contacto_nombre .

2.5 — tipos_seguro

Campo	Tipo	Constraints
id	Int	PK, autoincrement
nombre	String	@unique, NOT NULL
categoria	enum CategoriaSeguro	NOT NULL
descripcion	String	nullable

Índices: categoria .

2.6 — polizas

Campo	Tipo	Constraints	Notas
id	Int	PK, autoincrement	
numero_poliza	String	@unique, NOT NULL	

cliente_id	Int	FK → clientes.id, NOT NULL	
aseguradora_id	Int	FK → empresas_aseguradoras.id, NOT NULL	
tipo_seguro_id	Int	FK → tipos_seguro.id, NOT NULL	
cobertura_id	Int	FK → coberturas.id, NOT NULL	antes: cobertura: CoberturaTipo enum global. Ahora cobertura del catálogo específico del tipo de seguro.
pago_id	Int	FK → pagos.id SetNull, nullable	1 pago : N pólizas
estado	enum PolizaEstado	NOT NULL	
fecha_inicio_vigencia	Timestamptz	NOT NULL	
fecha_fin_vigencia	Timestamptz	NOT NULL, CHECK ≥ inicio	
suma_asegurada	Decimal(15,2)	NOT NULL	
prima_mensual	Decimal(15,2)	NOT NULL	Cuota mensual fija; ver §1.15
moneda	Char(3)	default 'ARS'	
modificado_por_id	Int	FK → usuarios.id SetNull, nullable	
modificado_en	Timestamptz	nullable	

Índices: cliente_id , aseguradora_id , tipo_seguro_id , cobertura_id , estado , fecha_fin_vigencia , pago_id , compuesto (estado, fecha_fin_vigencia) .

2.7 — siniestros

Campo	Tipo	Constraints	Notas
id	Int	PK, autoincrement	Identificador interno
poliza_id	Int	FK → polizas.id cascade, NOT NULL	
numero	String	@unique, NOT NULL	Identificador visible al cliente; ver §1.14
titulo	String	NOT NULL	

fecha_ocurrencia	Timestamptz	NOT NULL	
fecha_reporte	Timestamptz	NOT NULL, CHECK ≥ ocurrencia	
estado	enum SiniestroEstado	NOT NULL, default nuevo	5 estados; ver §3
modificado_por_id	Int	FK → usuarios.id SetNull, nullable	
modificado_en	Timestamptz	nullable	

Eliminado: fuente (solo se reciben por WhatsApp), leído (movido a siniestro_lecturas, ver §2.11). **Tabla puente eliminada:** siniestros_poliza. **Índices:** poliza_id, estado, fecha_reporte DESC.

2.8 — siniestro_documentos

Campo	Tipo	Constraints
id	Int	PK, autoincrement
siniestro_id	Int	FK → siniestros.id cascade, NOT NULL
tipo	enum DocumentoTipo	NOT NULL
nombre	String	NOT NULL
url	String	NOT NULL
mime_type	String	NOT NULL
tamano_bytes	Int	nullable

Eliminado: procesado_ia. **Índices:** siniestro_id.

2.9 — pagos

Campo	Tipo	Constraints	Notas
id	Int	PK, autoincrement	
cliente_id	Int	FK → clientes.id, NOT NULL	Quién inicia el pago (ver §1.5 y §6 obs. #2)
monto	Decimal(15,2)	NOT NULL	Total del batch
fecha_pago	Timestamptz	nullable	Se llena al validar
estado	enum PagoEstado	NOT NULL, default pendiente	
metodo_pago	enum MetodoPago	nullable	Enum controlado; ver §1.13

moneda	Char(3)	default 'ARS'	
modificado_por_id	Int	FK → usuarios.id SetNull, nullable	
modificado_en	Timestamptz	nullable	

Relación con pólizas: 1:N (1 pago agrupa N pólizas). La FK vive en `polizas.pago_id`. No hay tabla puente. **Eliminados:** `fecha_emision`, `periodo`, `monto_total` (calculado), `concepto` (vive implícito en cada póliza vinculada), `comprobante`, `cbu`. **Índices:** `cliente_id`, `estado`, `fecha_pago DESC`, `compuesto (cliente_id, estado)`.

2.10 — usuarios

Campo	Tipo	Constraints
id	Int	PK, autoincrement
dni	String	@unique, NOT NULL
email	String	@unique, NOT NULL
nombre_completo	String	NOT NULL
rol	enum Rol	NOT NULL

Eliminada: tabla `roles` (reemplazada por enum `Rol`).

2.11 — coberturas (nueva)

Catálogo de coberturas válidas por tipo de seguro. Reemplaza al enum global `CoberturaTipo`.

Campo	Tipo	Constraints
id	Int	PK, autoincrement
tipo_seguro_id	Int	FK → tipos_seguro.id cascade, NOT NULL
nombre	String	NOT NULL
descripcion	String	nullable

Constraint: `@@unique([tipo_seguro_id, nombre])` — el mismo nombre puede repetirse entre tipos distintos (ej: "responsabilidad_civil" en auto y en hogar son entradas distintas con su propio id). **Índices:** `tipo_seguro_id`.

2.12 — siniestro_lecturas (nueva)

Registra qué usuario leyó qué siniestro y cuándo. Sustituye al flag `siniestros.leido`.

Campo	Tipo	Constraints
siniestro_id	Int	FK → siniestros.id cascade, NOT NULL
usuario_id	Int	FK → usuarios.id cascade, NOT NULL

leido_en	Timestamptz	default now()
----------	-------------	---------------

PK compuesta: (siniestro_id, usuario_id) — un usuario lee un siniestro como mucho una vez (la lectura más reciente queda persistida; si se quisiera historial completo sería otra tabla). **Índices:** usuario_id .

3. Enums

```

ClienteTipo      → corporativo | persona
ClienteEstado    → activo | baja
PolizaEstado      → vigente | proxima | vencida | anulada | renovada
SiniestroEstado   → nuevo | pendiente_documentacion | en_tramite | cerrado |
rechazado
CategoriaSeguro   → auto | vida | hogar | salud | comercio | art | agricola | otros
MetodoPago        → transferencia | debito_automatiko | tarjeta_credito |
tarjeta_debito | efectivo | mercadopago | cheque | otro
DocumentoTipo     → img | pdf
PagoEstado        → pendiente | validado | rechazado
Rol               → productor | administrativo

```

Eliminado: CoberturaTipo (reemplazado por tabla coberturas , ver §2.11).

Los valores de CategoriaSeguro , MetodoPago y los nombres del catálogo coberturas son una propuesta basada en el dominio observable; ajustables si el negocio define otros.

4. Resumen estructural

Tablas eliminadas (8): poliza_cliente , poliza_empresa , tipo_poliza , cobertura_poliza , siniestros_poliza , pagos_polizas , roles . (La tabla coberturas original se rediseña como catálogo dependiente, ver §2.11.)

Tablas nuevas (2): coberturas (catálogo), siniestro_lecturas (estado por-usuario).

(Total: pasa de **17 tablas** → **12 tablas**.)

Campos eliminados clave:

- empresas_aseguradoras : color_hex , direccion , contacto_nombre
- polizas : fecha_emision , tarjeta_circulacion_poliza , numero_cuota , cobertura (reemplazado por cobertura_id)
- siniestros : fuente , descripcion_hechos , ai_summary , leido
- siniestro_documentos : procesado_ia , hash
- pagos : fecha_emision , periodo , monto_total , concepto , comprobante , cbu
- usuarios : password_hash (campo de implementación, no expuesto en el modelo de datos)

Campos transformados:

- Estados, fuente, tipo de doc, rol, categoria de seguro, método de pago → enum
- tamano (String) → tamano_bytes (Int)
- usuarios.dni (PK) → id_artificial PK + dni @unique
- tipos_seguro.nombre (PK) → id_artificial PK + nombre @unique

- Auditoría `created_at / updated_at` → `modificado_por_id` (FK → usuarios) + `modificado_en`
- `polizas.cobertura` (enum) → `polizas.cobertura_id` (FK → coberturas)
- `pagos.metodo_pago` (String) → `MetodoPago?` enum
- `siniestros.leido` (Boolean) → eliminado (movido a `siniestro_lecturas`)

Campos agregados:

- `clientes.tipo` (discriminador)
- `modificado_por_id` + `modificado_en` en `clientes` , `polizas` , `siniestros` , `pagos` , `empresas_aseguradoras`
- `polizas.moneda` , `pagos.moneda`
- `polizas.cobertura_id`
- `siniestro_documentos.url` , `mime_type`
- `tipos_seguro.categoria`

5. Próximos pasos (post-aprobación)

1. Reescribir `prisma/schema.prisma` reflejando este modelo.
2. Generar migración (`npx prisma migrate dev --name v3_coberturas_metodopago_lecturas`).
3. Adaptar `prisma/seed-data.ts` y `prisma/seed.ts` (catálogo de coberturas por tipo, `MetodoPago` enum, eliminar `leido`).
4. Adaptar `lib/data/_mappers.ts` y `lib/data/types.ts` .
5. Validar con `npx prisma validate` y correr seed.
6. Adaptar Server Actions y componentes UI que consumen `cobertura` , `metodo_pago` o `leido` .

6. Respuestas a correcciones recibidas

Las observaciones del corrector fueron evaluadas individualmente. Las que aportan a la consistencia del modelo se aplicaron; las que entran en conflicto con el dominio o con decisiones documentadas previamente se rechazaron con justificación. La intención es mostrar el criterio, no aceptar todo sin discusión.

#	Observación	Veredicto	Justificación
1	Invertir relación <code>pólizas↔pagos</code> (eliminar <code>polizas.pago_id</code> , agregar <code>pagos.poliza_id</code>)	Rechazada	El Glosario de la Entrega 1 define el Pago como <i>"transacción financiera única que agrupa el dinero enviado por un Cliente Corporativo hacia una Aseguradora específica en una fecha determinada"</i> . El pago es un batch que agrupa N pólizas del mismo cliente. Invertir a <code>pagos.poliza_id</code> (1 póliza : N pagos) rompe ese caso de uso: cada batch tendría que duplicarse para cada póliza incluida. Se mantiene <code>polizas.pago_id</code> .
2	Quitar <code>pagos.cliente_id</code> por redundante	Rechazada	Con <code>polizas.pago_id</code> , el cliente del pago no es accesible sin join. Además, semánticamente el Pago corporativo pertenece a un cliente desde su nacimiento: existe en estado pendiente antes de

			tener pólizas vinculadas (el cliente notifica que va a transferir antes de saber qué pólizas cubrirá). Eliminar <code>cliente_id</code> rompería esa secuencia. Se mantiene como atributo natural del Pago.
3	Documentar <code>polizas.prima_mensual</code>	Aceptada	Documentada en §1.15: cuota mensual fija congelada en la emisión; el monto del pago no es necesariamente $12 \times prima_mensual$.
4	<code>siniestros.leido</code> mezcla UI con dominio	Aceptada	Movido a tabla <code>siniestro_lecturas</code> (§2.12). La lectura se atribuye al usuario que abrió el siniestro y se conserva el <code>leido_en</code> .
5	<code>tipos_seguro</code> insuficiente, falta categoría	Aceptada	Agregado categoría: <code>CategoriaSeguro</code> (§1.9, §2.5). Permite filtrar y agrupar por rama del seguro sin depender del nombre libre.
6	<code>SiniestroEstado</code> con poca granularidad	Aceptada (moderada)	Expandido de 3 a 5 valores: <code>nuevo</code> , <code>pendiente_documentacion</code> , <code>en_tramite</code> , <code>cerrado</code> , <code>rechazado</code> . Se evita el exceso de microestados (ej. "pericia_programada") porque no aportan valor de negocio en el alcance actual y son fácilmente representables como notas dentro del estado <code>en_tramite</code> .
7	<code>CoberturaTipo</code> específico de auto contradice multi-tipo	Aceptada (crítica)	Reintroducida la tabla <code>coberturas</code> (<code>id</code> , <code>tipo_seguro_id</code> , <code>nombre</code>) (§2.11). <code>polizas.cobertura</code> pasa de enum a <code>cobertura_id</code> FK. El enum global <code>CoberturaTipo</code> se elimina. Cada tipo de seguro define su propio catálogo de coberturas válidas.
8	<code>pagos.metodo_pago</code> debe ser enum	Aceptada	Convertido en <code>MetodoPago?</code> (§1.13, §3). Mantiene nullable para el estado pendiente de un pago aún no materializado.
9	<code>siniestros.numero</code> sin documentar	Aceptada	Documentado en §1.14: identificador visible al cliente con formato <code>SIN-NNNNN</code> , generado por la aplicación y distinto del <code>id</code> interno.

Archivos a modificar

- `prisma/schema.prisma`
- `prisma/seed.ts`
- `prisma/seed-data.ts`
- `lib/data/_mappers.ts`
- `lib/data/types.ts`
- `lib/data/{clientes,polizas,siniestros,aseguradoras,pagos,kpis}.ts`
- Server Actions de pólizas (cobertura como FK), siniestros (lectura por usuario) y pagos (`metodo_pago` enum)
- Componentes UI que muestran cobertura, método de pago o estado de lectura de siniestros
- Mocks del chatbot (`chatbot/app/models.py` , `chatbot/app/mock_system.py` , `chatbot/app/seed.py`) para reflejar la nueva forma de cobertura y categoría